

The Right Ways to Address Cross-Site Request Forgery

Here's an overview of the key concepts, impact and effective mitigation strategies of cross-site request forgery, also known as CSRF.



CSRF (cross-site request forgery) was included in the OWASP (Open Worldwide Application Security Project) Top 10 list in 2013 but was removed from it in 2017 as the statistical data did not justify its place there. However, CSRF still impacts web applications a great deal.

The effect of a CSRF exploit varies from case to case. There are multiple factors that decide the severity of the exploit, which could be:

- Unauthorised actions [money transfer, account setting changes]
- Privilege escalation [admin access]
- Application integrity and confidentiality loss [data theft]
- Possible loss of reputation [negative image within users or communities]

SOP (Same-Origin Policy)

To understand why CSRF is successful, one needs to understand the Same-Origin Policy (SOP) used by browsers. The

latter follow SOP by default and only allow requests from the same origin. However, there is a business need for the user to make a cross-origin or cross-domain request to the application server. There are a lot of security concerns around allowing cross-domain requests. Initially, browser implementations used CORS (cross-origin resource sharing) to accommodate cross-domain requests while taking security concerns into consideration. Recent implementations have come up with a cookie attribute called 'SameSite'. Let us discuss CORS and SameSite in brief.

To address the need of cross-origin requests, CORS specifications are used, and browsers are made compliant with CORS specs. Application servers explicitly whitelist the trusted domains from where they can accept cross-origin requests and browsers are directed accordingly, using CORS-specific response headers.

Browsers are also implemented to consider the cookie attribute called 'SameSite'. These attribute values are set by the application server and direct the browser to decide if the domain cookies are to be included in the request or not. The browser decides for which cases the session cookies are to be sent for different types of requests to the server. All modern browsers support this attribute, which is an effective means of protection from CSRF attacks.

Role of the browser in a CSRF attack

The typical CSRF attack scenario is shown in Figure 1.

It is clear from this figure that the attacker takes advantage of the browser sending domain cookies for the server to validate the session authenticity. Therefore, the victim needs to be logged in to the same browser for a successful CSRF exploit.